TecPro

TECNOLOGIAS

Desenvolvimento, Apostilas e Cursos Online



Linguagem



Listas



Árvores



Complexidade

Algoritmos e Estruturas de Dados

Em C++, com comparações em Python

Algoritmos e Estruturas de Dados

Em C++, com comparações em Python

TecPro Tecnologias

Desenvolvimento, Apostilas e Cursos Online

2026

Todos os direitos reservados. Nenhuma parte desta publicação pode ser reproduzida, armazenada ou transmitida de qualquer forma, ou por qualquer meio, sem autorização prévia da TecPro Tecnologias.

Sumário

Capítulo 1 – Fundamentos de C++ para Estruturas de Dados **4**
1.1 Por que estudar estruturas de dados em C++? 4
1.2 Estrutura de um programa em C++ 4
1.3 Tipos, variáveis e vetores 5
1.4 Funções e passagem de parâmetros 6
Por valor: a função recebe uma cópia 6
Por referência: a função trabalha na variável original 6
Por referência constante: sem cópia e sem alteração 7
1.5 Ponteiros 8
1.6 Memória dinâmica: new e delete 9
1.7 Structs: agrupando dados 10
Uma prévia: o nó de uma lista encadeada 10
1.8 Classes: dados e comportamento juntos 11
1.9 Templates: estruturas para qualquer tipo 13
Erros Comuns 13

Capítulo 2 – Complexidade de Algoritmos **15**
2.1 Por que medir a eficiência? 15
2.2 Medir o tempo ou analisar o algoritmo? 15
2.3 Contando operações 16
2.4 Melhor caso, pior caso e caso médio 16
2.5 A notação Big O 17
A ideia: ficar com o termo que domina 17
A definição formal 17
2.6 As principais classes de complexidade 18
2.7 Analisando código passo a passo 19
Laço simples: O(n) 19
Laços aninhados: $O(n^2)$ 20
Laço que divide a entrada: O(log n) 20
Laço que divide dentro de um laço linear: O(n log n) 21
Recursão 21
2.8 Complexidade de espaço 22
Erros Comuns 23

Capítulo 1 – Fundamentos de C++ para Estruturas de Dados

Em Algoritmos I você aprendeu a base da programação: variáveis, tipos e operadores, condicionais e laços, funções, strings, vetores e matrizes. Também conheceu, de forma introdutória, listas, pilhas, filas, busca, ordenação e recursão. Agora vamos aprofundar: construir essas estruturas por dentro, de forma eficiente, e analisar o custo de cada operação.

Para isso, você precisa de algumas ferramentas do C++ que as estruturas de dados usam o tempo todo. Este capítulo faz duas coisas:

- **retoma rapidamente** o que você já conhece, agora escrito em C++: estrutura de um programa, tipos, vetores e funções (seções 1.2 a 1.4);
- **apresenta o que é novo** e indispensável daqui em diante: ponteiros, memória dinâmica, structs, classes e templates (seções 1.5 a 1.9).

Se você já se sente seguro com a primeira parte, pode passar por ela rapidamente. A segunda parte merece atenção total: é a base de todos os capítulos seguintes.

1.1 Por que estudar estruturas de dados em C++?

Em linguagens como Python, Java ou JavaScript, a memória é administrada automaticamente. Você cria uma lista, adiciona elementos e nunca precisa pensar onde eles estão guardados. Isso é ótimo para produtividade, mas esconde justamente o que queremos entender nesta disciplina: **como os dados são organizados na memória**.

Em C++, você enxerga e controla essa organização. Uma lista encadeada deixa de ser uma caixa-preta e passa a ser o que ela realmente é: um conjunto de blocos de memória, cada um apontando para o próximo. Quando você entende isso, passa a entender também por que uma operação é rápida ou lenta, e esse é o centro da análise de algoritmos.

Ao longo do livro, sempre que ajudar a entender um conceito, vamos mostrar como a mesma ideia aparece em Python. Assim você vê as duas faces da moeda: o que acontece *por dentro* (C++) e como isso costuma ficar *escondido* em uma linguagem de mais alto nível (Python).

1.2 Estrutura de um programa em C++

Todo programa em C++ começa a ser executado pela função `main`. Veja um programa completo:

```
#include <iostream>   // biblioteca de entrada e saída (cin, cout)
using namespace std;  // permite escrever cout em vez de std::cout

int main() {
    int idade;
    cout << "Digite sua idade: ";
    cin >> idade;                       // lê um valor do teclado
```

```
    cout << "Daqui a 10 anos voce tera " << idade + 10 << " anos" <<
    endl;
    return 0;                              // 0 indica que terminou sem
    erro
}
```

Alguns pontos importantes:

- `#include` traz uma biblioteca para o programa. A `iostream` oferece `cin` (entrada) e `cout` (saída).
- Todo comando termina com `;` e todo bloco fica entre `{` e `}`.
- C++ diferencia letras maiúsculas de minúsculas: `idade` e `Idade` são variáveis diferentes.
- O programa precisa ser **compilado** antes de rodar. Com o compilador g++, por exemplo: `g++ programa.cpp -o programa`.

Comparando com Python

Em Python, o mesmo programa não precisa de `main`, de tipos nem de compilação, porque é interpretado linha a linha:

```
idade = int(input("Digite sua idade: "))
print(f"Daqui a 10 anos voce tera {idade + 10} anos")
```

Em C++, o compilador verifica os tipos antes de o programa rodar. Muitos erros que em Python só aparecem durante a execução, em C++ aparecem já na compilação.

1.3 Tipos, variáveis e vetores

Uma variável é um espaço na memória com um nome e um tipo. O tipo define quanto espaço ela ocupa e que valores pode guardar.

Tipo	Guarda	Exemplo
`int`	números inteiros	`int qtd = 10;`
`double`	números com casas decimais	`double nota = 8.5;`
`char`	um caractere	`char letra = 'A';`
`bool`	verdadeiro ou falso	`bool ok = true;`
`string`	texto (precisa de `<string>`)	`string nome = "Ana";`

Um **vetor** (array) guarda vários valores do mesmo tipo, um ao lado do outro na memória. Os índices começam em 0:

```
int notas[5] = {7, 9, 6, 10, 8};   // 5 inteiros seguidos na memória

for (int i = 0; i < 5; i++) {
    cout << "Nota " << i << ": " << notas[i] << endl;
}
```

O vetor tem uma limitação que vai motivar vários capítulos deste livro: o **tamanho é fixo**. Se você declarou espaço para 5 notas e chegou uma sexta, não há onde colocá-la. Inserir um elemento no meio também é trabalhoso, porque todos os seguintes precisam ser deslocados uma posição. As estruturas que vamos construir, como as listas encadeadas, existem justamente para contornar essas limitações.

1.4 Funções e passagem de parâmetros

Uma função é um trecho de código com nome, que recebe dados (parâmetros), faz um trabalho e pode devolver um resultado. Funções evitam repetir código e deixam o programa mais fácil de ler e de manter.

```
// recebe os três coeficientes e diz se a equação tem raízes reais
bool temRaizReal(double a, double b, double c) {
    double delta = b * b - 4 * a * c;
    return delta >= 0;
}

// void: a função não devolve valor, só executa uma ação
void imprimirVetor(int v[], int tamanho) {
    for (int i = 0; i < tamanho; i++) {
        cout << v[i] << " ";
    }
    cout << endl;
}
```

O ponto mais importante para este livro é **como os parâmetros chegam à função**. Em C++ há três formas.

Por valor: a função recebe uma cópia

```
void dobrar(int x) {
    x = x * 2;          // altera só a cópia
}

int main() {
    int n = 5;
    dobrar(n);
    cout << n << endl;  // imprime 5: o n original não mudou
}
```

Por referência: a função trabalha na variável original

Basta acrescentar `&` depois do tipo do parâmetro. A função passa a usar a própria variável de quem chamou, sem cópia:

```
void dobrar(int &x) {
    x = x * 2;          // altera a variável original
}
```

```
void trocar(int &a, int &b) {
    int auxiliar = a;
    a = b;
    b = auxiliar;
}

int main() {
    int n = 5;
    dobrar(n);
    cout << n << endl;  // imprime 10

    int p = 1, q = 2;
    trocar(p, q);
    cout << p << " " << q << endl;  // imprime 2 1
}
```

Por referência constante: sem cópia e sem alteração

Quando o dado é grande (um texto longo, um vetor de milhares de itens) e a função só precisa ler, use `const &`. Você evita a cópia, que custa tempo e memória, e o compilador garante que a função não altera o original:

```
#include <string>

int contarVogais(const string &texto) {
    int total = 0;
    for (char c : texto) {
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')
        {
            total++;
        }
    }
    return total;
}
```

Comparando com Python

Python não deixa você escolher. Números e textos se comportam como se fossem passados por valor, porque não podem ser alterados (são *imutáveis*). Já listas e dicionários são passados de forma que a função pode modificá-los:

```
def dobrar(x):
    x = x * 2          # cria um novo número só dentro da função

def adicionar(lista):
    lista.append(99)   # modifica a lista original

n = 5
dobrar(n)
```

```
print(n)       # 5

numeros = [1, 2]
adicionar(numeros)
print(numeros) # [1, 2, 99]
```

Em C++ essa decisão é sua e fica escrita na assinatura da função, o que torna o comportamento explícito.

1.5 Ponteiros

Toda variável ocupa uma posição na memória, e cada posição tem um **endereço**, como o número de uma casa em uma rua. Um **ponteiro** é uma variável que guarda um endereço: em vez de guardar um valor, ele aponta para onde o valor está.

Dois operadores fazem todo o trabalho:

- `&variavel` devolve o **endereço** da variável;
- `*ponteiro` acessa o **valor** que está no endereço guardado (isso se chama *desreferenciar*).

```
int idade = 20;
int *p = &idade;      // p guarda o endereço de idade

cout << idade << endl;  // 20
cout << p << endl;      // um endereço, por exemplo 0x61fe14
cout << *p << endl;     // 20: o valor que está no endereço

*p = 21;                // altera idade através do ponteiro
cout << idade << endl;  // 21
```

Uma forma de visualizar o que aconteceu na memória:

Variável	Endereço	Conteúdo
`idade`	`0x61fe14`	`21`
`p`	`0x61fe18`	`0x61fe14` (aponta para `idade`)

Um ponteiro que não aponta para nada deve valer `nullptr`. Você vai usar muito esse valor: é ele que marca o fim de uma lista encadeada ou um galho vazio de uma árvore. Antes de desreferenciar um ponteiro, verifique se ele não é nulo:

```
int *q = nullptr;   // não aponta para nada

if (q != nullptr) {
    cout << *q << endl;  // só acessa se for seguro
}
```

Referência ou ponteiro?

Os dois permitem trabalhar com a variável original, mas têm diferenças. Uma **referência** (`int &r = x;`) é um apelido: precisa nascer ligada a uma variável e nunca muda de alvo. Um **ponteiro** pode começar vazio (`nullptr`) e pode passar a apontar para outro lugar. Por isso as estruturas de dados usam ponteiros: um nó de uma lista precisa poder apontar para *nenhum* nó, ou mudar de vizinho quando você insere ou remove elementos.

1.6 Memória dinâmica: new e delete

As variáveis declaradas dentro de uma função ficam em uma área chamada **pilha de execução** (*stack*). Elas são criadas quando a função começa e destruídas automaticamente quando ela termina. Isso é prático, mas tem um limite: você não consegue criar, durante a execução, quantos elementos quiser e mantê-los vivos depois que a função acabar.

Para isso existe a memória dinâmica, ou **heap**. Com `new`, você pede ao sistema um espaço que continua existindo até você mesmo liberá-lo com `delete`:

```
int *p = new int;     // reserva espaço para um int no heap
*p = 42;
cout << *p << endl;   // 42
delete p;             // devolve o espaço
p = nullptr;          // boa prática: não deixar o endereço antigo

// vetor com tamanho decidido durante a execução
int n;
cout << "Quantos valores? ";
cin >> n;
int *valores = new int[n];   // n inteiros no heap
for (int i = 0; i < n; i++) {
    valores[i] = i * i;
}
delete[] valores;            // vetor: libera com delete[]
```

Toda estrutura de dados dinâmica deste livro funciona assim: cada novo elemento é criado com `new` e, quando sai da estrutura, é liberado com `delete`.

Regra de ouro da memória dinâmica

Para cada `new` deve existir exatamente um `delete` (e para cada `new[]`, um `delete[]`). Se você esquecer o `delete`, o espaço fica preso até o programa terminar: é o chamado **vazamento de memória** (*memory leak*). Se fizer `delete` duas vezes, ou usar o ponteiro depois do `delete`, o comportamento é imprevisível e o programa pode travar.

Comparando com Python

Em Python não existe `delete` para liberar memória. O próprio interpretador percebe quando um objeto não é mais usado por ninguém e o remove: é a **coleta de lixo** (*garbage collection*). Isso evita vazamentos e ponteiros inválidos, mas tem um custo:

você perde o controle de quando a memória é liberada e de quanto espaço cada estrutura realmente ocupa.

1.7 Structs: agrupando dados

Uma `struct` junta vários dados relacionados em um único tipo novo. Em vez de ter variáveis soltas para nome, idade e matrícula de um aluno, você cria o tipo `Aluno`:

```
struct Aluno {
    string nome;
    int matricula;
    double media;
};

int main() {
    Aluno a;
    a.nome = "Ana";          // o ponto acessa cada campo
    a.matricula = 2026001;
    a.media = 8.7;

    Aluno *p = &a;
    cout << p->nome << endl; // com ponteiro, usa-se a seta ->
}
```

Repare no operador `->`: quando você tem um **ponteiro** para uma struct, `p->nome` é o mesmo que `(*p).nome`. Ele vai aparecer em praticamente todo o código dos próximos capítulos.

Uma prévia: o nó de uma lista encadeada

Uma struct pode ter um campo que é ponteiro para o próprio tipo. Essa ideia simples é a base das listas encadeadas e das árvores:

```
struct No {
    int valor;
    No *proximo;   // aponta para o próximo nó (ou nullptr)
};

int main() {
    // cria três nós no heap
    No *a = new No{10, nullptr};
    No *b = new No{20, nullptr};
    No *c = new No{30, nullptr};

    // liga os nós: a -> b -> c
    a->proximo = b;
    b->proximo = c;

    // percorre a cadeia a partir do primeiro nó
    for (No *atual = a; atual != nullptr; atual = atual->proximo) {
```

```
        cout << atual->valor << " ";   // 10 20 30
    }
    cout << endl;

    delete a;
    delete b;
    delete c;
}
```

Os três nós podem estar em qualquer lugar da memória, não precisam estar lado a lado como num vetor. O que os mantém em ordem são os ponteiros. Por isso, para inserir um valor no meio, basta mudar dois ponteiros, sem deslocar nenhum elemento. No Capítulo 3, essa ideia vira uma lista completa.

1.8 Classes: dados e comportamento juntos

Uma classe vai além da struct: ela reúne os dados e as funções que trabalham com eles, e controla quem pode acessar o quê. Esse controle se chama **encapsulamento**. Os dados ficam em `private`, protegidos de alterações indevidas, e o acesso acontece por funções públicas, os **métodos**.

Como exemplo, veja uma classe que resolve o problema do tamanho fixo dos vetores: um vetor que cresce sozinho quando enche.

```
class VetorDinamico {
private:
    int *dados;        // vetor no heap
    int tamanho;       // quantos elementos estão em uso
    int capacidade;    // quantos cabem antes de crescer

    void crescer() {
        capacidade = capacidade * 2;
        int *novo = new int[capacidade];
        for (int i = 0; i < tamanho; i++) {
            novo[i] = dados[i];      // copia os elementos antigos
        }
        delete[] dados;              // libera o vetor antigo
        dados = novo;
    }

public:
    // construtor: executado quando o objeto é criado
    VetorDinamico() {
        capacidade = 2;
        tamanho = 0;
        dados = new int[capacidade];
    }

    // destrutor: executado quando o objeto deixa de existir
    ~VetorDinamico() {
        delete[] dados;
```

```
    }

    void adicionar(int valor) {
        if (tamanho == capacidade) {
            crescer();
        }
        dados[tamanho] = valor;
        tamanho++;
    }

    int obter(int indice) const {
        return dados[indice];
    }

    int quantidade() const {
        return tamanho;
    }
};

int main() {
    VetorDinamico v;
    for (int i = 1; i <= 10; i++) {
        v.adicionar(i * 10);   // cresce sozinho: 2, 4, 8, 16 posições
    }
    for (int i = 0; i < v.quantidade(); i++) {
        cout << v.obter(i) << " ";
    }
    cout << endl;
}   // aqui o destrutor libera a memória automaticamente
```

Repare em três ideias que vão se repetir ao longo do livro:

- **Construtor** (`VetorDinamico()`): prepara o objeto quando ele nasce. Tem o mesmo nome da classe e não tem tipo de retorno.
- **Destrutor** (`~VetorDinamico()`): libera os recursos quando o objeto deixa de existir. Assim, quem usa a classe não precisa se lembrar de chamar `delete`.
- **Métodos** `const`: `obter` e `quantidade` apenas consultam o objeto, e o `const` garante que não alteram nada.

Todas as estruturas dos próximos capítulos (lista, pilha, fila, árvore) serão classes com esse formato: dados privados, métodos públicos para as operações e um destrutor que libera os nós.

Comparando com Python

A lista do Python (`list`) funciona exatamente como o `VetorDinamico`: por dentro, ela reserva espaço sobrando e, quando enche, cria um espaço maior e copia os elementos. Você só não vê esse trabalho. A mesma classe em Python ficaria assim:

```
class Contador:
    def __init__(self):        # construtor
```

```
        self._total = 0        # o _ indica "uso interno"

    def incrementar(self):
        self._total += 1

    def valor(self):
        return self._total
```

Em Python, o `private` é só uma convenção (o sublinhado), e não existe destrutor para liberar memória, porque a coleta de lixo cuida disso.

1.9 Templates: estruturas para qualquer tipo

O `VetorDinamico` só guarda `int`. Se você precisar de um vetor de `double` ou de `string`, teria de copiar a classe inteira trocando o tipo. Os **templates** resolvem isso: você escreve o código uma vez, com um tipo genérico `T`, e o compilador gera a versão de cada tipo usado.

```
// função genérica: funciona com int, double, string...
template <typename T>
T maior(T a, T b) {
    return (a > b) ? a : b;
}

// struct genérica: o nó pode guardar qualquer tipo
template <typename T>
struct NoGenerico {
    T valor;
    NoGenerico<T> *proximo;
};

int main() {
    cout << maior(3, 7) << endl;                          // 7
    cout << maior(2.5, 1.5) << endl;                      // 2.5
    cout << maior(string("ana"), string("bia")) << endl;  // bia

    NoGenerico<string> *n = new NoGenerico<string>{"oi", nullptr};
    cout << n->valor << endl;
    delete n;
}
```

A biblioteca padrão do C++ (STL) já traz estruturas prontas feitas com templates, como `vector`, `list`, `stack`, `queue` e `map`. Neste livro vamos construir as nossas versões, porque o objetivo é entender como elas funcionam por dentro. Depois de entender, use as da STL nos seus projetos: são testadas e otimizadas.

Erros Comuns

- **Usar um ponteiro não inicializado.** Declarar `int *p;` e depois fazer `*p = 5;` escreve em um endereço aleatório. Inicialize sempre: com um endereço válido

ou com `nullptr`.

- **Desreferenciar `nullptr`.** Acessar `p->valor` quando `p` é nulo derruba o programa. Em listas e árvores, confira se o ponteiro não é nulo antes de usá-lo.
- **Esquecer o `delete`.** Cada `new` sem o `delete` correspondente é memória perdida. Em classes, libere no destrutor.
- **Usar `delete` em vez de `delete[]`.** O que foi criado com `new[]` deve ser liberado com `delete[]`.
- **Usar o ponteiro depois do `delete`.** O espaço já foi devolvido, e o endereço antigo não é mais seu. Depois de liberar, faça `p = nullptr;`.
- **Esperar que uma função altere um parâmetro passado por valor.** Se a função precisa modificar a variável de quem chamou, use referência (`&`) ou ponteiro.
- **Acessar um vetor fora dos limites.** Em um vetor de 5 posições, os índices vão de 0 a 4. C++ não avisa quando você acessa `v[5]`: o programa simplesmente lê ou estraga outra área da memória.

Capítulo 2 – Complexidade de Algoritmos

Dois programas podem resolver o mesmo problema e dar a mesma resposta, mas um terminar em um piscar de olhos e o outro levar horas. A diferença quase nunca está no computador: está no algoritmo. Neste capítulo você vai aprender a medir a eficiência de um algoritmo sem precisar executá-lo, usando a notação Big O. Essa ferramenta vai acompanhar você em todos os capítulos seguintes: para cada estrutura de dados, vamos perguntar quanto custa inserir, buscar e remover.

2.1 Por que medir a eficiência?

Imagine um sistema com o cadastro de 1 milhão de clientes. Para encontrar um cliente pelo CPF, você pode olhar um por um, do primeiro ao último. No pior caso, são 1 milhão de comparações. Se o cadastro estiver ordenado, dá para fazer como numa lista telefônica: abrir no meio, ver se o CPF está antes ou depois e descartar metade a cada passo. Assim, bastam cerca de 20 comparações.

Os dois métodos funcionam, mas um é 50 mil vezes mais rápido que o outro. E a diferença aumenta conforme o cadastro cresce. É por isso que analisamos algoritmos: para escolher, antes de programar, a solução que continua viável quando os dados crescem.

A eficiência pode ser medida em dois recursos:

- **Tempo**: quantas operações o algoritmo faz. É a análise mais comum e o foco deste capítulo.
- **Espaço**: quanta memória extra o algoritmo precisa. Veremos na seção 2.8.

2.2 Medir o tempo ou analisar o algoritmo?

A forma mais direta de saber se um programa é rápido é executá-lo e cronometrar. Em C++, a biblioteca `chrono` faz isso:

```
#include <chrono>

long long somar(int n) {
    long long soma = 0;
    for (int i = 1; i <= n; i++) {
        soma += i;
    }
    return soma;
}

int main() {
    auto inicio = chrono::steady_clock::now();
    long long resultado = somar(100000000);
    auto fim = chrono::steady_clock::now();

    auto ms = chrono::duration_cast<chrono::milliseconds>(fim -
    inicio).count();
    cout << "Resultado: " << resultado << endl;
    cout << "Tempo: " << ms << " ms" << endl;
```

```
}
```

Essa **análise experimental** é útil, mas tem limites. O tempo medido depende do processador, do sistema operacional, do compilador, de outros programas abertos e até dos dados usados no teste. Rodando o mesmo código em outra máquina, o número muda. E, para medir, você precisa ter o algoritmo pronto.

A **análise de complexidade** olha para o algoritmo em si. Em vez de medir segundos, ela conta **quantas operações** o algoritmo faz em função do tamanho da entrada. O resultado vale para qualquer máquina e pode ser obtido antes de escrever o código. As duas análises se completam: a de complexidade ajuda a escolher o algoritmo, e a experimental ajusta os detalhes quando o tempo é crítico.

2.3 Contando operações

O primeiro passo da análise é definir o **tamanho da entrada**, que chamamos de n. Ele depende do problema:

Problema	Tamanho da entrada (n)
Ordenar ou buscar em um vetor	quantidade de elementos
Operações em uma lista ou árvore	quantidade de nós
Algoritmos em grafos	número de vértices e de arestas
Processar um texto	quantidade de caracteres

Depois, contamos as operações que importam. Veja a função que soma os elementos de um vetor:

```
int somarVetor(int v[], int n) {
    int soma = 0;               // 1 vez
    for (int i = 0; i < n; i++) {
        soma += v[i];           // n vezes
    }
    return soma;                // 1 vez
}
```

A atribuição inicial e o `return` acontecem uma vez; a soma dentro do laço acontece n vezes. Contando também as comparações e incrementos do `for`, chegamos a algo como $t(n) = 3n + 3$ operações. O número exato depende de como se conta, e é aí que entra a ideia mais importante do capítulo: **o que interessa não é o valor exato, e sim como $t(n)$ cresce quando n cresce**. Se o vetor dobrar de tamanho, o trabalho dobra. Esse comportamento é o que a notação Big O captura.

2.4 Melhor caso, pior caso e caso médio

Para muitos algoritmos, o número de operações depende não só do tamanho, mas também do **conteúdo** da entrada. A busca sequencial é o exemplo clássico:

```
// devolve a posição do valor procurado, ou -1 se não encontrar
int buscaSequencial(int v[], int n, int procurado) {
    for (int i = 0; i < n; i++) {
        if (v[i] == procurado) {
            return i;
        }
    }
    return -1;
}
```

- **Melhor caso**: o valor está na primeira posição. Uma comparação basta, não importa o tamanho do vetor.
- **Pior caso**: o valor está na última posição ou não existe. São n comparações.
- **Caso médio**: supondo que o valor esteja no vetor e que qualquer posição seja igualmente provável, em média percorremos metade do vetor, cerca de $n/2$ comparações.

O **pior caso** é o mais usado, por um motivo prático: ele é uma **garantia**. Se o pior caso de um algoritmo é aceitável, ele nunca vai decepcionar. O melhor caso costuma ser otimista demais para servir de base a uma decisão, e o caso médio exige saber como os dados se distribuem, o que nem sempre é possível.

2.5 A notação Big O

A notação Big O descreve como o tempo de um algoritmo cresce, deixando de lado os detalhes que não mudam esse crescimento. Dizemos que a busca sequencial é $O(n)$ (lê-se "ó de ene"): no pior caso, o tempo cresce de forma proporcional ao tamanho da entrada.

A ideia: ficar com o termo que domina

Suponha que um algoritmo faça $t(n) = 3n^2 + 5n + 20$ operações. Veja quanto cada parte contribui:

n	$3n^2$	$5n$	20	**parte de $3n^2$**
10	300	50	20	81%
100	30.000	500	20	98,3%
1.000	3.000.000	5.000	20	99,8%

Quando n cresce, o termo n^2 domina completamente. As constantes (o 3, o 5, o 20) dependem de detalhes de implementação e da máquina. Por isso, a notação Big O fica só com o termo dominante, sem a constante: esse algoritmo é $O(n^2)$.

A definição formal

Uma função $t(n)$ é $O(f(n))$ se existem constantes positivas c e k tais que

$$t(n) \leq c \cdot f(n) \quad \text{para todo } n \geq k.$$

Em palavras: a partir de um certo tamanho de entrada (k), o tempo do algoritmo nunca passa de um múltiplo (c) de $f(n)$. No exemplo acima, com $c = 4$ e $k = 8$, temos $3n^2 + 5n + 20 \leq 4n^2$ para todo $n \geq 8$; logo $t(n)$ é $O(n^2)$.

Regras práticas do Big O

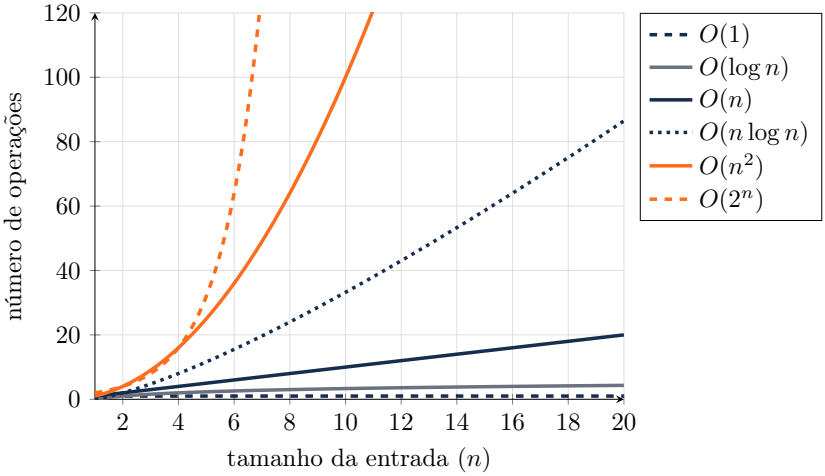
- **Descarte as constantes**: $O(3n)$ é $O(n)$; $O(n^2/2)$ é $O(n^2)$.
- **Fique com o termo de maior crescimento**: $O(n^2+n)$ é $O(n^2)$; $O(n+\log n)$ é $O(n)$.
- **Trechos em sequência se somam**: um laço $O(n)$ seguido de outro $O(n^2)$ resulta em $O(n^2)$.
- **Trechos aninhados se multiplicam**: um laço de n voltas dentro de outro de n voltas resulta em $O(n \cdot n) = O(n^2)$.

2.6 As principais classes de complexidade

Quase todos os algoritmos que você vai encontrar se encaixam em poucas classes:

Classe	Nome	Exemplo
$O(1)$	constante	acessar `v[i]`; inserir no início de uma lista encadeada
$O(\log n)$	logarítmica	busca binária em vetor ordenado
$O(n)$	linear	busca sequencial; somar um vetor
$O(n \log n)$	linearítmica	merge sort; heapsort
$O(n^2)$	quadrática	bubble sort; comparar todos os pares
$O(n^3)$	cúbica	multiplicação simples de matrizes
$O(2^n)$	exponencial	testar todos os subconjuntos de um conjunto

O gráfico mostra como essas funções crescem. Repare como $O(\log n)$ quase não sai do chão, enquanto $O(2^n)$ dispara logo no início:



Para sentir a diferença na prática, imagine um computador que faz **1 milhão de operações por segundo**. Quanto tempo cada classe levaria?

Classe	$n = 1.000$	$n = 1.000.000$
$O(\log n)$	10 microssegundos	20 microssegundos
$O(n)$	1 milissegundo	1 segundo
$O(n \log n)$	10 milissegundos	20 segundos
$O(n^2)$	1 segundo	cerca de 11,6 dias
$O(2^n)$	mais que a idade do universo	impossível

A tabela explica por que a escolha do algoritmo importa mais que a velocidade do computador. Um processador mil vezes mais rápido transforma os 11,6 dias do $O(n^2)$ em cerca de 17 minutos, mas um algoritmo $O(n \log n)$, na máquina antiga, resolve o mesmo problema em 20 segundos.

2.7 Analisando código passo a passo

Com as regras da seção 2.5, dá para descobrir a complexidade lendo o código. Veja os padrões mais comuns.

Laço simples: O(n)

```
// encontra o maior valor do vetor: percorre cada elemento uma vez
int maiorValor(int v[], int n) {
    int maior = v[0];
    for (int i = 1; i < n; i++) {
        if (v[i] > maior) {
            maior = v[i];
        }
```

```
    }
    return maior;
}
```

Laços aninhados: O(n²)

```
// verifica se o vetor tem algum valor repetido, comparando todos os
pares
bool temRepetido(int v[], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (v[i] == v[j]) {
                return true;
            }
        }
    }
    return false;
}
```

O laço interno começa em `i + 1`, então ele dá $n - 1$ voltas, depois $n - 2$, e assim por diante. A soma é $n(n-1)/2$ comparações no pior caso. Descartando a constante $1/2$ e o termo menor, a função é $O(n^2)$.

Laço que divide a entrada: O(log n)

```
// quantas vezes n pode ser dividido por 2 até chegar a 1?
int contarDivisoes(int n) {
    int passos = 0;
    while (n > 1) {
        n = n / 2;
        passos++;
    }
    return passos;
}

int main() {
    cout << contarDivisoes(1000) << endl;     // 9
    cout << contarDivisoes(1000000) << endl;  // 19
}
```

Sempre que um algoritmo **descarta metade** do problema a cada passo, ele é logarítmico. Multiplicar a entrada por mil acrescenta só uns 10 passos. É o segredo da busca binária (Capítulo 7) e das árvores balanceadas (Capítulo 5).

Laço que divide dentro de um laço linear: O(n log n)

```
int exemploNLogN(int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {           // n vezes
        for (int j = n; j > 1; j = j / 2) { // log n vezes
            total++;
        }
    }
    return total;
}
```

Esse é o padrão dos bons algoritmos de ordenação, como o merge sort (Capítulo 8): cada um dos $\log n$ níveis de divisão faz um trabalho linear.

Recursão

Em Algoritmos I você viu que uma função recursiva chama a si mesma. Para analisá-la, conte quantas chamadas acontecem e quanto trabalho cada uma faz.

```
// fatorial: n chamadas, cada uma com trabalho constante -> O(n)
long long fatorial(int n) {
    if (n <= 1) return 1;
    return n * fatorial(n - 1);
}

// Fibonacci recursivo: cada chamada gera outras duas -> O(2^n)
long long fibRecursivo(int n) {
    if (n <= 1) return n;
    return fibRecursivo(n - 1) + fibRecursivo(n - 2);
}

// Fibonacci com laço: guarda só os dois últimos valores -> O(n)
long long fibIterativo(int n) {
    long long anterior = 0, atual = 1;
    for (int i = 0; i < n; i++) {
        long long proximo = anterior + atual;
        anterior = atual;
        atual = proximo;
    }
    return anterior;
}

int main() {
    cout << fatorial(10) << endl;      // 3628800
    cout << fibRecursivo(30) << endl;  // 832040, após mais de 1
    milhão de chamadas
    cout << fibIterativo(30) << endl;  // 832040, com apenas 30 voltas
}
```

O Fibonacci recursivo é um ótimo alerta: o código é curto e elegante, mas recalcula os mesmos valores muitas vezes. Para `fibRecursivo(50)`, seriam bilhões de chamadas.

A versão com laço resolve em 50 passos. No Capítulo 9 você vai ver a programação dinâmica, uma técnica que conserta exatamente esse tipo de desperdício.

Comparando com Python

Em Python, operações que parecem instantâneas podem esconder um laço inteiro. Saber a complexidade delas evita surpresas:

```
nomes = ["ana", "bia", "caio", "davi"]

print("davi" in nomes)   # O(n): percorre a lista até achar
nomes.append("eva")      # O(1) em média: coloca no fim
nomes.insert(0, "zoe")   # O(n): desloca todos os elementos
nomes.pop()              # O(1): remove do fim
nomes.pop(0)             # O(n): desloca todos os elementos

conjunto = set(nomes)
print("davi" in conjunto)  # O(1) em média: usa tabela hash
```

Uma linha como `if x in lista` dentro de um laço transforma um algoritmo que parecia $O(n)$ em $O(n^2)$. Em C++, como você escreve os laços, esse custo fica visível no próprio código.

2.8 Complexidade de espaço

A mesma notação mede a **memória extra** que o algoritmo usa, além da própria entrada.

- `somarVetor` e `fibIterativo` usam só algumas variáveis, não importa o tamanho de n: espaço $O(1)$.
- Um algoritmo que cria uma cópia do vetor para trabalhar nela usa espaço $O(n)$.
- O `fatorial` recursivo também usa espaço $O(n)$, mesmo sem criar vetores: cada chamada que ainda não terminou ocupa um espaço na pilha de execução, e há n chamadas abertas ao mesmo tempo.

Muitas vezes é possível trocar espaço por tempo: guardar resultados já calculados gasta memória, mas evita refazer contas. Essa troca é a base da programação dinâmica.

Para saber mais: Ômega e Theta

Além do Big O, existem outras duas notações. A notação Ω (ômega) indica um **limite inferior**: o algoritmo faz *pelo menos* aquela quantidade de trabalho. A notação Θ (theta) indica um **limite exato**: o crescimento é *exatamente* daquela ordem, nem mais nem menos. Na prática, quando alguém diz "a busca sequencial é $O(n)$", costuma querer dizer que o pior caso cresce exatamente como n. Neste livro, seguindo o uso comum, usaremos o Big O.

Erros Comuns

- **Confundir Big O com tempo em segundos.** $O(n)$ não diz quanto tempo o programa leva, e sim como esse tempo cresce. Um algoritmo $O(n)$ pode ser mais lento que um $O(n^2)$ para entradas pequenas.
- **Manter constantes e termos menores.** Escrever $O(2n + 10)$ está tecnicamente certo, mas o correto é simplificar para $O(n)$.
- **Somar quando deveria multiplicar.** Dois laços de n voltas *um depois do outro* dão $O(n)$; *um dentro do outro* dão $O(n^2)$.
- **Esquecer o custo das operações de biblioteca.** Chamar uma função que percorre a estrutura inteira (como `in` em uma lista do Python) dentro de um laço multiplica a complexidade.
- **Analisar só o melhor caso.** Dizer que a busca sequencial é $O(1)$ porque às vezes o valor está na primeira posição esconde o comportamento real. Use o pior caso como referência.
- **Ignorar o espaço da recursão.** Uma função recursiva com n chamadas abertas ao mesmo tempo usa memória $O(n)$, e com n muito grande pode estourar a pilha de execução.